Latest updates: https://dl.acm.org/doi/10.1145/3714393.3726520

RESEARCH-ARTICLE

To the Best of Knowledge and Belief: On Eventually Consistent Access Control

FLORIAN JACOB, Karlsruhe Institute of Technology, Karlsruhe, Baden-Wurttemberg, Germany

HANNES HARTENSTEIN, Karlsruhe Institute of Technology, Karlsruhe, Baden-Wurttemberg, Germany

Open Access Support provided by:

Karlsruhe Institute of Technology

PDF Download
3714393.3726520.pdf
21 March 2026
Total Citations: 0
Total Downloads: 385

Published: 04 June 2025

Citation in BibTeX format

CODASPY '25: Fifteenth ACM Conference on Data and Application Security and Privacy
June 4 - 6, 2025
PA, Pittsburgh, USA

Conference Sponsors:
SIGSAC

CODASPY '25: Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy (June 2024)
https://doi.org/10.1145/3714393.3726520
ISBN: 9798400714764

To the Best of Knowledge and Belief: On Eventually Consistent Access Control

Florian Jacob
florian.jacob@kit.edu
Karlsruhe Institute of Technology
Karlsruhe, Germany

Hannes Hartenstein
hannes.hartenstein@kit.edu
Karlsruhe Institute of Technology
Karlsruhe, Germany

Abstract

We are used to the conventional model in which access control is provided by a trusted central entity or by a set of distributed entities that coordinate to mimic a central entity. Authorization decisions are based on a single, append-only total ordering of all actions, including policy updates, which leads to strong consistency guarantees. Recent systems based on conflict-free replicated data types (CRDTs) break with this conceptual model to gain fundamental advantages in latency, availability, resilience, and Byzantine fault tolerance. These systems replace up-front coordination with subsequent reconciliation of decentralized authorization decisions and policy updates. One of these is the Matrix group communication system, whose massive public sector deployments in Europe necessitate timely characterization of the underlying alternative conceptual model. Similarly to eventually consistent replication in CRDTs, we define 'eventually consistent access control' and present its consequences. Our model postulates thinking in two orderings of actions with different consequences for authorization: a partial ordering for storage, where the past of an action is final knowledge, and a total ordering for execution, where the past of an action is a mutable belief.

CCS Concepts

• **Security and privacy** → **Access control**; **Distributed systems security**; • **Software and its engineering** → **Consistency**; *Publish-subscribe / event-based architectures*; • **Information systems** → **Distributed storage**; • **Computer systems organization** → *Distributed architectures*; *Availability*; **Dependable and fault-tolerant systems and networks**; *Reliability*.

Keywords

Access Control, Eventual Consistency, Authorization, Monotonicity, Decentralized Systems, Byzantine Fault Tolerance, CRDTs, Matrix

ACM Reference Format:
Florian Jacob and Hannes Hartenstein. 2025. To the Best of Knowledge and Belief: On Eventually Consistent Access Control. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy (CODASPY '25), June 4–6, 2025, Pittsburgh, PA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3714393.3726520

This work is licensed under a Creative Commons Attribution 4.0 International License.
CODASPY '25, Pittsburgh, PA, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1476-4/2025/06
https://doi.org/10.1145/3714393.3726520

1 Introduction

Coordination takes time, and results in dependencies to other entities. In line with the famous quote popularized by Grace Hopper, "It's easier to ask forgiveness than it is to get permission." [6, 19], system designers often strive to make time-critical actions independent of the latency to other entities. In the realm of data consistency, this principle limits the achievable consistency to models such as eventual consistency [1], but enables to tolerate an arbitrary number of Byzantine-faulty entities taking part in the system [14]. But what does this principle mean in the realm of access control consistency? In context, Hopper's quote *is specifically on* (organizational) authorization decisions and reduction of their coordination and latency. Access control found in group communication and collaboration systems based on Byzantine-tolerant CRDTs [14] and local-first ideals [15], like Matrix [25] or Keyhive [5], follows this idea. While the Keyhive research project started in 2024, Matrix has grown since 2014 to be a security-critical topic: nation states like France, Sweden, and Germany operate private federations for their public sectors [10], the United Nations International Computing Center has switched to Matrix as communication platform provided to UN organizations [17], and more than 100 000 000 users on more than 100 000 servers are found in the public federation.

In this paper, we refer to the conventional conceptional model of access control as linearizable access control (LAC): the access control architecture acts as a single, logically centralized entity that stores, decides on, and enforces all access control policies and policy updates. LAC can be implemented by a distributed system as coordination-based linearizable access control (CLAC): a set of system entities coordinates on policy updates and authorization decisions to simulate a central entity. In contrast, we define eventually consistent access control (ECAC) to describe systems whose entities implement a logically decentralized access control architecture: system entities autonomously store, decide on, and enforce access control policies and their updates to their best of knowledge and belief on the overall current system state. The challenge of ECAC is to deal with the concurrent nature of decentralized policy updates and authorization decisions, in particular for revocations and in presence of Byzantine entities. To ensure that access control policies and decisions eventually converge, up-front coordination among system entities is replaced with subsequent reconciliation:

CLAC:	**ECAC:**
(1) Coordination	(1) Decision
(2) Decision	(2) Access
(3) Access	(3) Reconciliation

At first glance, giving up on CLAC semantics by replacing coordinated decisions with accountable best-effort decisions seems

like a prohibitive trade-off to make. However, ECAC variants are found in many deployed systems that prioritize latency, availability, or fault tolerance: for example, offline payments in planes or offline withdrawals at ATMs prioritize availability over consistency, and reconciliation allows to audit for overdraft later. Electronic door locks also typically provide best-effort service, as the risk of unauthorized people getting in due to a stale policy might be more acceptable to business operation than the risk of authorized people not getting in due to network outage, also potentially stopping them from fixing the network outage in the first place. Furthermore, PKI certificate validation in web browsers makes a local decision and updates trusted (root) certificates only from time to time.

In this paper, we identify the invariants of access control under Byzantine eventual consistency, and study the trade-off between fundamentally improved latency, availability, and resilience, at the cost of a drastic change in application guarantees. The goal is to *understand and characterize* ECAC, not to claim 'superiority' over CLAC. Most related to our work is the research line on distributed authorizations (e.g. [16]). Our work differs in investigating the concurrency of *multiple* entities deciding authorization for *the same* replicated object. We provide three main contributions:

- The ECAC model is our propositional answer to what kind of access control is achievable in eventually consistent systems. Its properties serve both as application guarantees and as necessary conditions for implementations.
- An assessment of the consequences of decentralized policy updates and authorization decisions shown under partitioning, equivocation, and backdating in the ECAC model.
- Matrix and other practical systems already represent proofs by example of ECAC's implementability. We provide an abstract algorithm open to scrutiny based on Matrix, and verify that it fulfills ECAC's necessary conditions.

In this paper, we are going from implementation to abstraction. We first present an overview of Matrix in Section 2 together with basics on conflict-free replicated data types. The problem statement of ECAC is presented in Section 3. We formalize the properties of the ECAC model in Section 4. We assess the ECAC model using a set of scenarios in Section 5, and summarize the key characteristics of ECAC. A formalization of an abstract ECAC algorithm is given in Appendix A. We include related work in-place where relevant.

2 Matrix Fundamentals

We make use of Matrix [25] as a real-world 'instance' of a yet-to-be-defined ECAC model. Matrix is a decentralized system based on conflict-free replicated data types (CRDTs, [12, 22]) that provides group communication and data storage. CRDTs are a class of coordination-free [3] (also called wait-free [9]) replication algorithms that work on the premise that concurrent updates can be joined to a common state that is an advancement on previous states, without needing coordination or user interaction to resolve conflicts. Matrix stands out from other systems by expecting Byzantine entities, and its emphasis on decentralized access control. Due to the autonomy of entities, CRDT-based decentralized systems may tolerate an arbitrary fraction of Byzantine faulty entities, making them immune to Sybil attacks [4, 13, 14].

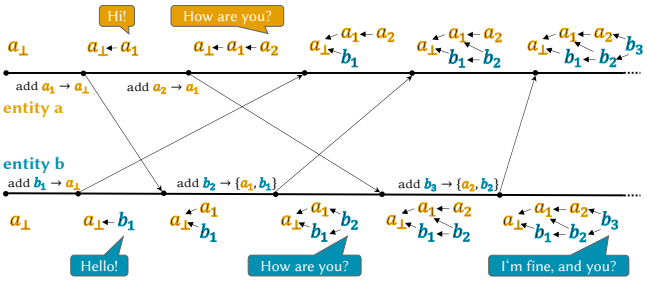


Figure 1: Matrix-based chat example for replicating a chronicle, i.e., a causally-ordered set of events. An event x_n is created by entity x, and points to its direct predecessors. The unique event without predecessors $x_\perp$ is called the genesis event, an event x_n has a longest path of length n to $x_\perp$. Both entities a, b concurrently add new events to the chronicle. Correct entities independently verify authorization of an event both before creation and before adding it to their local state.

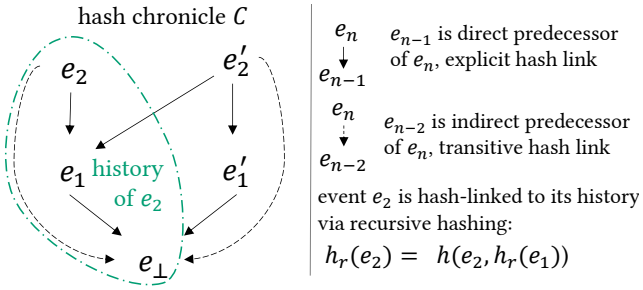


Figure 2: Example hash chronicle C. The causal order on events is divided into explicit hash links to direct predecessors and implicit, transitive links to other predecessors.

The state of a Matrix *communication group* (called "room" in Matrix) consists mainly of its shared communication history, but also includes metadata like membership, attributes of group members and the group itself, as well as access control policies and permissions. Instead of storing mutable group state directly, Matrix stores a grow-only, partially-ordered set of immutable group state change *events*, which are executed to derive the current group state. The core of Matrix is the CRDT-based replication of these sets (which we call *chronicles*) among all entities in the group [13], exemplified in Fig. 1. Events encapsulate a subject's action on an optional object, taking place in a communication group as environment. For example, a chat message is an event in the group by the sender subject, with an action of type `cht` with content `"Hi!"`, and no object.

As shown in Fig. 2, events include hash links to their direct predecessors. We refer to hash-linked event sets as *hash chronicles* [13] (called "event graphs" in Matrix). Hash links are chosen at the discretion of the event's creator — testifying that the creator knew *at least* the predecessors, but they may be aware of additional concurrent events. The recursive hash $h_r(e)$ of an event e hashes e with the concatenation of the recursive hashes of its direct predecessors. Thereby, the recursive hash links an event to its *history*, i.e., the chronicle subset related to e, which enables receiving entities to

detect and re-request missed events in a process called *backfilling*. By using a collision-resistant hash function, Byzantine entities cannot create different events with the same recursive hash, whereby correct entities will eventually know about all events. Together with digital signatures, recursive hash linking ensures authenticity, integrity, and eventual consistency of chronicle replication.

Similar to email servers, Matrix servers act as trusted representatives for their users. In this work, we assume that a server has only one user, and treat server and user as single *entity*. Every entity performs an independent authorization decision before adding an event to its chronicle. Authorizations in Matrix are expressed using the Level- and Attribute-based Access Control (LeABAC) model [11], as shown in Fig. 3. Initial authorizations are at the group creator's discretion. Authorizations revolve around a function *lvl* that maps entities and types of event actions to permission levels. Events that change levels must define *lvl* for all entities and action types, to allow multiple atomic changes and to prevent undesired results on concurrent changes. For an event e to be authorized, its creator subject $e.sbj$ must be authorized for a level greater or equal than its type of action $e.act$. Sending an event also requires a subject's group membership attribute to be *mbr*:*IN*. Additionally, in case the event changes authorizations, it must either grant authorization to its object for a level less or equal than the subject's level, or revoke authorization for a level that is less than the subject's level. Also, subjects can only perform actions on objects that have a lower level than themselves.

Authorization is checked before storing *and* before executing an event, on different bases: An event is only *stored* if it is authorized by the state derived from executing the *immutable set of its predecessors*. An event is only *executed*, i.e., its encapsulated state change is only applied, if the event is authorized by the state derived from *totally ordering all events of the current chronicle*. The process of totally ordering the events of the chronicle is called "state resolution" in Matrix. Since the various Matrix servers involved in a room might have differing views on the current state of the chronicle, one might wonder what guarantees such an approach can give and what requirements have to be imposed on the state resolution algorithm. These aspects are further discussed in the following section and precisely defined in Section 4.

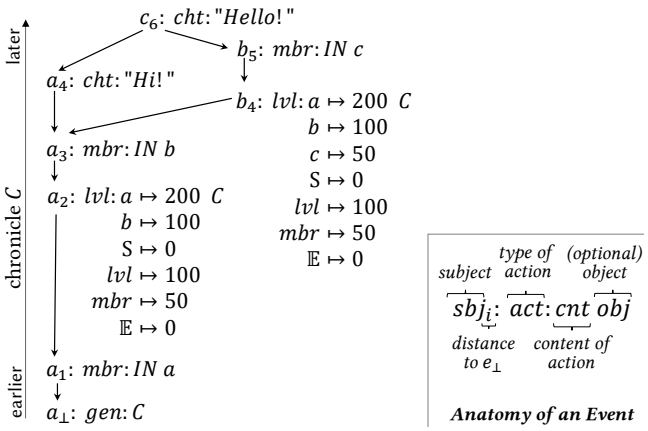


Figure 3: Example of Level- and Attribute-based Access Control in Matrix, in which attenuated authorizations flow from group creator a over b to c. Genesis event $a_\perp$ grants authorization to its creator a to send a first membership event a_1, declaring a to be `IN` the group, as well as a first permission level assignment a_2, assigning $a \mapsto 200$ and $b \mapsto 100$. So while b is authorized both for membership changes ($mbr \mapsto 50$) and level assignments ($lvl \mapsto 100$), it cannot act against a. In a_3, a adds b to the group. Using its granted authorizations, b assigns $c \mapsto 50$ in b_4, otherwise repeating the previous `lvl`, and adds c in b_5. The chat message c_6 : *cht*:"*Hello*!" of c is indirectly authorized by a granting authorization to b.

3 ECAC Problem Statement

Decentralized access control solutions typically assume that either every object is controlled by a single respective entity that decides authorization for that object [16], or that all entities sharing object ownership coordinate to perform authorization decisions. The key challenge of ECAC, however, is to provide decentralized access control on *shared* replicated data, but *without* coordination.

We use the following terminology to describe the problem further. Access control architectures are characterized by the placement of their crucial components: the Policy Enforcement Points (PEPs) that intercept actions, Policy Decision Points (PDPs) that issue the authorization decision, and the components that provide the basis for decision-making in terms of policies (Policy Retrieval Point, PRP) and attributes of subjects, actions, and objects (Policy Information Point, PIP) [23]. System entities create *events* by taking the conjunction of subject, i.e., themselves, action, and object. We focus on actions that change state, in particular administrative actions that change access control state. The following access control architectures implement execution monitoring [21]: based on policies and known previous events, an *execution monitor* decides whether a new event is authorized, and prevents event processing if not. Authorized events are appended to the append-only *audit log*. The order of events in the audit log acts as logical timestamp of events. The execution of the audit log's events leads to the system's *app state*, which includes the access control state. Events and their set of predecessors are *immutable*, i.e., they cannot be changed after creation. *Finality* means unchangeable after a point in time, immutability means finality after creation. A set is *grow-only* if its elements are immutable and cannot be removed, but new elements can be added. A partially-ordered set is *append-only* if it is grow-only and new elements can only be appended, i.e., a new element is either larger or concurrent to any other set element, but not smaller.

While audit logs are fundamental to ECAC, we also describe LAC and CLAC based on audit logs to highlight the differences in the conceptual models. The LAC, CLAC, and ECAC approaches differ in their access control architecture, as contrasted in Fig. 4, and especially in the way events are ordered in their audit log.

In LAC (Fig. 4a), there is only one instance of every access control architecture component, which represents both the centralized execution monitor ideal as well as its practical implementation. The LAC model is based on a totally-ordered audit log. We call this order the central order, i.e., the order in which events become visible for the central entity. Events are executed in central order to eliminate concurrency, which indirectly resolves conflicts due to concurrent

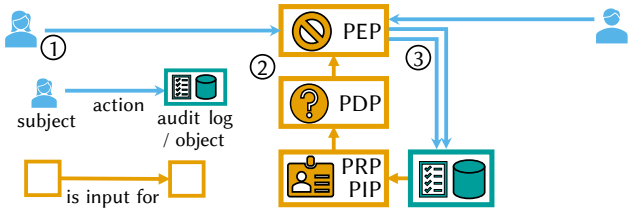
policy updates. The authorization decision for an event is based on the app state from executing all its predecessors. As the audit log is append-only, predecessor sets are immutable, and thereby, authorization decisions are immutable as well.

CLAC is the usual way of distributing an access control architecture (Fig. 4b): The components of the central entity in LAC are instantiated once on every distributed entity, and coordination is required to ensure that all entities issue the same decisions based on the same policy information. Thus, coordination provides audit log consensus, to simulate LAC's centrally-ordered audit log.

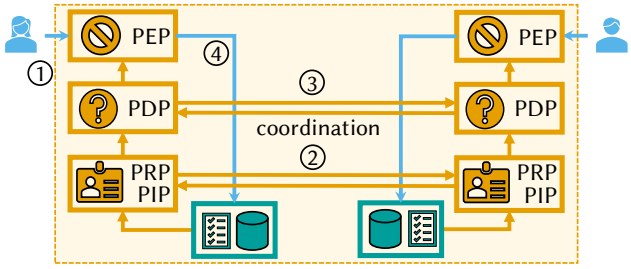
While the placement of ECAC components (Fig. 4c) is the same as in CLAC, the challenge in terms of defining a conceptual model is the different communication pattern of components. Instead of coordinating with all other entities up-front to find consensus, ECAC is based on coordination-free access control decisions that are correct to the best of knowledge and belief locally available to the deciding entity. Based on previous work on access control for weakly consistent databases like CRDTs [20, 28], the problem at hand is to find a conceptual model in line with the properties of Byzantine CRDTs, like the Matrix approach described in Section 2.

In ECAC, the audit log provides a partial (causal) order of policy updates and authorization decisions due to concurrency, and is reconciled during favorable network conditions. Unlike LAC's central order, the partial order in ECAC does not naturally resolve conflicts among concurrent events that affect each other, like authorization revocations. We are going to define the ECAC conceptual model (c.f. Fig. 5) using two orders instead of one central order: an append-only partial order for storage as audit log, and a grow-only total order for execution. The execution order is derived using the lexicographic order that defines execution priority among events as a refining sequence of comparison criteria, i.e., their lexicographic product. For example, authorization revocations can be prioritized, or subjects with more permissions get to act before subjects with less. Using the lexicographic order to resolve concurrency, entities extend the causal order via topological sorting, resulting in the topological order. While the topological order is a grow-only total order, it is not append-only: New events can appear anywhere in the order, as long as they do not violate causality. Causal and topological orders result in two notions of authorization in ECAC: We call an event *causally authorized* if it is authorized by its causal predecessors, and *topologically authorized* if it is authorized by its topological predecessors. An event that is authorized by its causal predecessors is not necessarily authorized by its topological predecessors as seen by the following example: If event b_1 is entity b adding entity c as member, but event a_1 is entity a concurrently revoking the authorization of b, then both events are causally authorized, but if event a_1 comes first in topological order, b_1 becomes topologically unauthorized, and entity c is not a member.

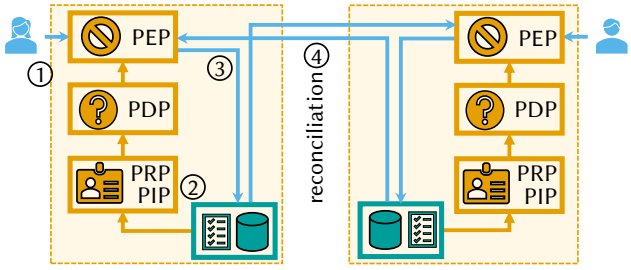
When defining ECAC, we want to protect the integrity of a replicated data structure in a Byzantine environment by performing coordination-free decentralized access control. The challenge is to find safety and liveness properties that make up a conceptual model for eventually consistent access control but that do not weaken the availability, scalability, or resilience qualities of Byzantine fault tolerant CRDTs. Specifically, the model must allow to tolerate an arbitrary fraction of Byzantine faulty entities under an asynchronous timing assumption and provide availability under partition.



(a) In LAC, actions of subjects are intercepted (1) by the Policy Enforcement Point (PEP), which implements a centralized execution monitor together with the Policy Information / Retrieval / Decision Point (PDP / PRP / PIP). Based on input by PRP & PIP, the PDP decides on the authorization of the action (2). Finally, to take effect on its object, the action is forwarded to the audit log (3).



(b) In CLAC, the PEP, PDP, and PRP/PIP are distributed, but still implement a centralized execution monitor via coordination. To decide authorization of an intercepted action (1), the PRP/PIPs coordinate to determine current policies and policy information (2), and the PDPs coordinate to reach consensus on the central order of events and access decision (3), before the action is forwarded (4).



(c) In ECAC, decisions are uncoordinated and autonomous, but implement a decentralized execution monitor via an eventually consistent audit log. To decide on the authorization of an intercepted action (1), PRP/PIP derive policies resp. policy information from the audit log for the PDP to decide to its best of knowledge and belief (2). The decision and basis for decision-making is recorded in the audit log, and the action is applied on the object (3). Audit logs are reconciled after the action took effect (4).

Figure 4: Comparison between Linearizable Access Control (LAC), Coordination-based Linearizable Access Control (CLAC), and Eventually Consistent Access Control (ECAC).

We proceed by consulting distributed systems theory [1, 3, 8] on an abstraction of Matrix, made up of three CRDT components [13]:

(1) the hash chronicle that stores a causally-ordered event set
(2) the topological event order derived by topological sorting
(3) the app state derived from topological event execution.

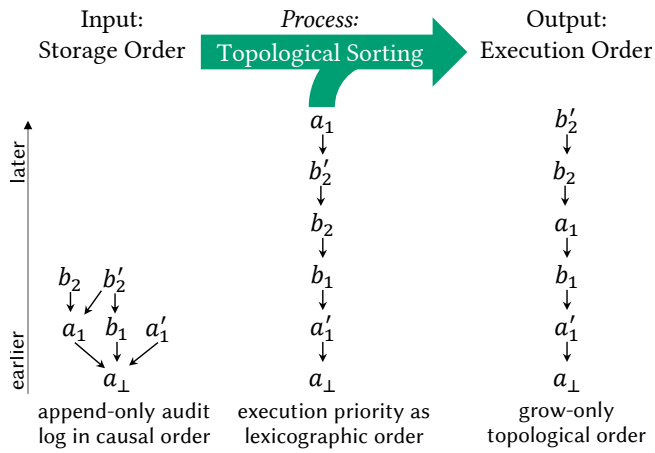


Figure 5: ECAC is based on three different orders of events: The causal storage order, which servers as input for topological sorting using the lexicographic prioritization order, to get the topological execution order as output. In the append-only causal order, the past of any included event is final, but events may be concurrent. The total lexicographic order, defined as part of the ECAC algorithm, reflects prioritization among events. Topological sorting resolves causal concurrency using the lexicographic order, leading to the topological order, a total, grow-only order. While the causal order is straightforward, the challenge of ECAC implementations is to define a lexicographic order and topological sorting so that concurrency conflicts are resolved in a way that neither compromises security nor application invariants.

4 ECAC Model

We now present a model of eventually consistent access control, consisting of a set of safety and liveness properties that act as both guarantees to the application as well as necessary conditions on ECAC algorithms. We present the ECAC conceptual model in two parts: First, we cover the properties of the underlying eventually consistent data type, and then the properties of the eventually consistent authorizations based on the data type. We introduce symbols and notations in-place, but list all in Table 1.

4.1 Eventually Consistent Data Type

The data type ensures local availability of policies and policy information, whereby its properties are necessary for a coordination-free authorization decision. Intuitively, eventual consistency is a consistency model that guarantees that over time, the state of correct system entities converges, to eventually reach a common, consistent state. Eventual consistency was originally defined in [24], consisting of the two properties that a) each data type update is eventually propagated to all system entities, and b) that non-commutative data type updates are executed in the same order by all system entities [1]. We assume that all functions executed by entities terminate, e.g., by using the total functional programming model [26] that achieves guaranteed termination by restriction to total functions and well-founded recursion. Because the execution order of commutative updates does not affect the resulting state by definition,

S	set of correct system entities
S^+	set of all system entities
$\mathbb{E}$	set of all events
$\mathbb{E}_z$	set of authorization events
$e.act$	type of action of event $e \in \mathbb{E}$
$e.obj$	object of event $e \in \mathbb{E}$, may be $\perp$ to signify no object
vis_s	set of events visible to $s \in S$
C_s	local chronicle, causal order of events stored by $s \in S$
$C_\cup$	global chronicle, the union of all local chronicles
$e^{\leq_E}$	set of predecessors (downward closure) of event $e \in \mathbb{E}$ in event set $E \subseteq \mathbb{E}$, $e^{\leq_E} = \{\hat{e} \in E \mid \hat{e} \leq_E e\}$
T_s	topological order derived from C_s of $s \in S$
L	lexicographic priority order of events, defined by the algorithm implementing ECAC
$x(T)$	app state set as set of events resulting from executing totally-ordered event set T
$x^{<e}(E)$	app state set resulting from executing the topological order of event set E up to, but not including event e
$z(X, e)$	Boolean, $\top$ if event $e \in \mathbb{E}$ is authorized by app state X

Table 1: Symbols and notations used in the ECAC model.

those properties guarantee that eventually, every system entity will be in the same state. A weaker notion of eventual consistency, "if no new updates are made [...], eventually all accesses will return the last updated value", was later popularized by Vogels [27], but a variation of the original definition gained traction under the name of strong eventual consistency as the consistency model for conflict-free replicated data types (CRDTs) [1, 2, 22]. Later, it was proven that the same definition originally conceived for crash fault environments also works in Byzantine environments, while needing different mechanisms to ensure its properties [14]. As recently suggested [1], we use the original definition of eventual consistency applied to CRDTs, consisting of the properties *eventual visibility* and *convergence* as follows:

Eventual Visibility (Liveness) An update visible for any correct entity is eventually visible for all correct entities.

Convergence (Safety) Entities that see the same set of updates have equivalent state.

Note that convergence is based on the *set* of updates, i.e., applies regardless of differences in update visibility order between entities. In our context of ordered sets of events, the properties that comprise eventual consistency become part of the ECAC model as *eventual event visibility* and *event set convergence*. We define $\mathbb{E}$ as set of all valid events, S as the set of correct system entities, and vis_s as the set of all events visible to entity s. We write C_s for the chronicle state of entity s, i.e., the causally ordered event set of s, and T_s for the topologically ordered event set of s. To execute a totally-ordered set of events T, we write $x(T) \subseteq T_s$, which returns an event set that describes the resulting app state.

Eventual Event Visibility An event visible for any correct entity is eventually visible for all correct entities.

$$\forall a, b \in S\colon e \in vis_a \implies \Diamond e \in vis_b \tag{1}$$

Event Set Convergence Correct entities seeing the same events have the same chronicle, topological order, and app state.

$$\forall a, b \in S\colon vis_a = vis_b \implies \tag{2}$$

$$C_a = C_b \wedge T_a = T_b \wedge x(T_a) = x(T_b) \tag{3}$$

We now define invariants that characterize the causally- and topologically-ordered event sets of correct entities at any given time. The causal event storage C_s of a correct entity $s \in S$ needs to satisfy *chronicality*, i.e., it must be a downward-closed, partially-ordered set directed at $e_\perp$. The topological order T_s derived from C_s of a correct entity $s \in S$ must fulfill *topological totality*, i.e., be a total order. In addition, T_s must fulfill *causal consistency* and *lexicographic consistency*, i.e., follow the causal order given by the chronicle C_s, and when ambiguous, fall back to the lexicographic order L, as defined by the ECAC algorithm implementing the model. If an event e_1 is a causal predecessor of event e_2 in a partially-ordered event set E, we write $e_1 \leq_E e_2$. If two events e_1 and e_1' are unordered in E, e.g., because they are causally concurrent, we write $e_1 \parallel_E e_1'$. A partially-ordered event set E is a subset of E' if both the set and the order are subsets of each other, $E \subseteq E' \iff \forall e \in E, e' \in E' : e \in E' \wedge (e \leq_E e' \implies e \leq_E' e')$. We write $C_\cup = \bigcup_{s \in S} C_s$ for the global chronicle, i.e., the union of all local chronicles.

Chronicality The local set C_s is always a chronicle, i.e., a partially-ordered event set, Eq. (4), that is downward-closed, Eq. (5), and directed at the global minimum $e_\perp$, Eq. (6).

$$\forall s \in S, \forall a, b \in C_s\colon \quad a \leq_{C_s} b \vee b \leq_{C_s} a \vee a \parallel_{C_s} b \tag{4}$$

$$\forall s \in S, \forall a \in C_s, \forall c \in C_\cup\colon \quad c \leq_{C_\cup} a \implies c \in C_s \tag{5}$$

$$\forall s \in S, \forall a \in C_s\colon \exists e_\perp \in C_s\colon \quad e_\perp \leq_{C_\cup} a \tag{6}$$

Topological Totality The topological order T_s is total.

$$\forall s \in S, \forall a, b \in T_s\colon a \leq_{T_s} b \vee b \leq_{T_s} a \tag{7}$$

Causal Consistency The topological order T_s preserves causality and contains the same events as C_s.

$$\forall s \in S, \forall e \in \mathbb{E}\colon C_s \subseteq T_s \wedge e \in C_s \iff e \in T_s \tag{8}$$

Lexicographic Consistency The topological order T_s orders causally concurrent events in accordance with the lexicographic order L.

$$\forall s \in S\colon \forall a, b \in C_s\colon a \parallel_{C_s} b \wedge a \leq_{T_s} b \implies a \leq_L b \tag{9}$$

We now define properties that characterize the evolution of the causally- and topologically-ordered events sets of correct entities over time. We are especially interested in monotonicity and immutability properties, as they hold independently of differences in event visibility orders [8]. We prime variables to denote a future state of the unprimed variable, e.g., local chronicle C_s evolves into C_s'. Monotonicity properties are safety properties that demand that if a future set of visible events vis_s' is greater or equal than the current set vis_s, then a derived value, like the chronicle, is also greater or equal than the current value. Immutability properties are the specialization where the derived value stays equal. As vis is grow-only, monotonically derived values represents certain knowledge that is not fallible in light of new information. For chronicles, we demand that the set of causal predecessors of any event must be immutable, which we formalize as *causal predecessor immutability*. In addition, we demand that the next chronicle state must include all previous events, which we formalize as *chronicle monotonicity*. We also require *topological monotonicity*, i.e., observing and ordering new events must not remove or change the order of old events.

Causal Predecessor Immutability The set of predecessors $e^{\leq C_s}$ of any event e in a chronicle C_s is immutable.

$$\forall s \in S, \forall e \in C_s\colon vis_s \subseteq vis_s' \implies e^{\leq C_s} = e^{\leq C_s'} \tag{10}$$

Chronicle Monotonicity The chronicle of a correct entity evolves monotonically, i.e., after observing new events, a correct entity inflates its local chronicle C_s to C_s' only by adding new events and their causal relations, while preserving old events and their causal predecessors.

$$\forall s \in S\colon vis_s \subseteq vis_s' \implies C_s \subseteq C_s' \tag{11}$$

Topological Monotonicity The topological order of a correct entity evolves monotonically, i.e., after observing new events, a correct entity inflates its topological order T_s to T_s' only by adding new events and new relations.

$$\forall s \in S\colon vis_s \subseteq vis_s' \implies T_s \subseteq T_s' \tag{12}$$

Note that topological monotonicity implies *topological predecessor monotonicity*, $e^{\leq T_s} \subseteq e^{\leq T_s'}$, but not immutability – in contrast to LAC, new topological predecessors can always become visible.

4.2 Eventually Consistent Authorization

To complete the ECAC model, we now define properties regarding authorizations derived from and applied to the different event sets of the data type. Authorizations determine the actions that a subject is allowed to execute on which objects. Policies define the relation between policy information, like attributes of subjects and objects, and the subjects' authorizations. Authorizations therefore depend on both the specification of policies as well as the required policy information. We assume that policy information is encoded as attributes of subjects and objects. Events take place in a communication group as their environment. Applicable policies in a communication group are encoded as its attributes, policy updates have the communication group itself as object. We assume that all actions take place inside the same communication group, and thereby do not explicitly specify the environment on definition.

We speak of authorization events $\mathbb{E}_z \subseteq \mathbb{E}$ as the subset of events that potentially change authorizations, i.e., policy or policy information update events. Authorization events can grant an authorization for causally succeeding events, or revoke an authorization in causally succeeding as well as causally concurrent events.

The base requirement for eventually consistent access control is that authorizations are independent of the order in which authorization events become visible, whereby entities will not end up in a split-brain situation where convergence is impossible due to conflicting events by Byzantine entities. We formalize these requirements as follows: *eventual authorization visibility* means that an authorization event visible for one correct entity is eventually visible for all correct entities. *Authorization convergence* means that two entities that see the same authorization events conclude the same authorizations, and thereby perform the same authorization decisions. As the data type does not distinguish between authorization events and other events, those properties directly follow from eventual event visibility and event set convergence.

We now define invariants that characterize the role of causal and topological authorization in ECAC. An event is causally authorized if it is authorized by the app state resulting from executing its causal predecessors. An event is topologically authorized if it is authorized by the app state resulting from executing its topological predecessors. We write $z(X, e)$ for the function that determines whether the event e is authorized based on the state set X, returning a truth value from the Boolean lattice $\mathcal{B} = \{\bot, \top\}$. Whether $z(X, e)$ verifies causal authorization or topological authorization depends on whether X is the result of executing causal or topological predecessors. The genesis event is the only event authorized by the empty set, $z(\emptyset, e_\bot) = \top$. To speak about different state sets for authorization, we define the shorthand notation for executing all events in the topological order of the partially-ordered set $E \subseteq T_s$ up to, but not including event e, $x^{<e}(E) = x(T_s \cap e^{<_E})$.

From causal predecessor immutability follows *causal authorization immutability*: $vis_s \subseteq vis'_s \implies x^{<e}(C_s) = x^{<e}(C'_s) \implies z(x^{<e}, C_s)) = z(x^{<e}, C'_s)$. As soon as an event's predecessor set is known, the entity can issue an immutable causal authorization decision that is independent of the order in which events became visible, which is why we say that causal authorization is eventually consistent access control *to the entity's best of knowledge*. Correct entities only send causally authorized events, as they would not use an authorization they do not possess. While faulty entities can send causally unauthorized events, those events will never pass causal authorization at correct entities. We thereby demand that chronicle replication verifies causal authorization: correct entities must only store events in their chronicle that are causally authorized, which we formalize as *storage authorization*.

Due to topological predecessor monotonicity, topological authorization decisions are mutable and never final, which is why we say that topological authorization is eventually consistent access control *to the entity's best of belief*. Specifically, authorization revocations are the cause of non-monotonicity of topological authorization decisions: a correct entity cannot state anything about the future topological authorization of an event currently deemed as topologically authorized or unauthorized, as learning about causally concurrent but topologically earlier authorization revocation events can always lead to changes in the topological authorization decision. On event execution, topologically unauthorized events must be ignored, but kept in case the become (re-)authorized later. Without revocations, we would end up with a monotonic protection system [7], for which a decentralized implementation could provide a strong, unconditional "topological authorization monotonicity" guarantee, akin to causal authorization immutability. However, in Byzantine environments, we need the possibility to revoke authorizations from Byzantine entities, e.g., if a member of a group chat posts spam messages and ought to get its group membership and messaging authorizations revoked. While correct entities only send events that are topologically authorized to the best of their belief, i.e., their topological order T_s, due to causally concurrent revocation events, we cannot demand that every event in any T_s must be topologically authorized. Instead, *execution authorization* prescribes that only topologically authorized events can have an effect on the app state set $x(T)$ resulting from executing T. In addition, *app state authorization* prescribes that all events in the app state set $x(T)$ must be topologically authorized.

Storage Authorization Every event e stored in state C_s of a correct replica is causally authorized.

$$\forall s \in S\colon e \in C_s \implies z(x^{<e}(C_s), e) \tag{13}$$

Execution Authorization Executed events are authorized by their topological past, i.e., topologically unauthorized events in T_s have no effect on the app state set $x(T_s)$.

$$\forall s \in S\colon x(T_s) = x(\{e \in T_s \mid z(x^{<e}(T_s), e)\}) \tag{14}$$

App State Authorization Every event e included in the app state $x(T_s)$ is topologically authorized.

$$\forall s \in S\colon e \in x(T_s) \implies z(x^{<e}(T_s), e) \tag{15}$$

Up until now, all discussed properties only indirectly influence the exposed app state of the system. We established that app state must be topologically authorized, but that topological authorization is mutable due to concurrent authorization revocations. Now, we combine the eventually consistent data type and eventually consistent authorization to characterize the evolution of the exposed app state itself, namely the app state set $x(T_s)$ of correct entities over time. We require that app state must evolve monotonically if there are no revocations, and that revocations are the only source of non-monotonicity, which we formalize as *app state confluence*. Specifically, an authorization revocation event concurrent with an event that uses the revoked authorizations are in conflict, and lead to an order dependency where entities decide differently depending on the order in which they see the events. An entity which first sees the revocation event and then the usage event exposes monotonically-evolving app state, as the usage event is not executed. However, an entity which first sees the usage and then the revocation must roll back its execution result to an earlier event, which is not monotonic, but allowed under app state confluence. Still, due to eventual event visibility and event set convergence, entities eventually decide "as if they had known" of concurrent events, and the app state eventually converges.

Events fall in two categories: either they have no object ($e.obj = \bot$), these events are the constituents of the communication history, or they have an object, these events describe updates of attribute $e.act$ on object $e.obj$). On execution, a topologically authorized event with a given ($e.act$, $e.obj$) will replace topologically earlier events with the same ($e.act$, $e.obj$). The app state set resulting from the execution $x(T)$ of a totally ordered set of events T thereby may only contain a single event per ($e.act$, $e.obj$) combination. This event describes the state of attribute $e.act$ on object $e.obj$ after T.

App State Confluence If event e in app state set $x(T_s)$ and event e' in app state set $x(T'_s)$ have equal *act* and *obj*, and T'_s is derived from a later visible events than T_s, the successor e' is either equal to or topologically larger than the predecessor e, or the predecessor lost its topological authorization.

$$\forall s \in S, \forall e \in x(T_s), \forall e' \in x(T'_s), \tag{16}$$

$$e.act = e'.act, e.obj = e'.obj\colon \tag{17}$$

$$vis_s \subseteq vis'_s \implies e \leq_{T'_s} e' \vee \neg z(x^{<e}(T'_s), e) \tag{18}$$

4.3 Classification of ECAC Model Properties

Eventual event visibility is the liveness property of ECAC, other properties are safety properties. We now characterize the safety

properties regarding invariant confluence and monotonicity. In essence, these properties characterize the independence of event visibility ordering. Monotonicity and immutability properties describe entity state evolution *while* events become visible in arbitrary order, whereas the other ECAC safety properties describe state *after* arbitrary-order visibility.

Coordination-free distributed systems cannot provide arbitrary services. They are limited to the concept of invariant confluence [3]: An invariant is confluent if the following implication holds: When every entity ensures the invariant locally based on its partial knowledge of events, the invariant also holds globally based on complete knowledge of all events. For example, an invariant that a set is grow-only is confluent, while an invariant that limits the maximum size of the set is not. As part of eventual consistency, convergence is an invariant-confluent property – otherwise, CRDTs would require coordination to ensure it. Convergence is ensured by every correct entity applying the same total function [26] on the set of updates that they see, and thus convergence holds globally. The same line of reasoning also applies to all ECAC properties: they do not rely on coordination, but only on total functions that derive entity state, like the current chronicle, from the unordered set of visible events.

In general, decentralized systems cannot hide the inherent nondeterminism of distributed systems in form of concurrency and reordering. The CALM theorem [8] ("Consistency as Logical Monotonicity") characterizes the subclass of invariant-confluent problems and algorithms whose outputs are also invariant to reordering of inputs as exactly the class of *monotonic* problems and algorithms. A problem or algorithm is monotonic if the following implication holds: When their input is greater or equal than another input, the output is also be greater or equal. Monotonicity alleviates nondeterminism induced by the system's distributed nature, whereby this subclass is especially suited for coordination-free decentralized systems. Monotonicity is stronger than invariant confluence, and thereby, the monotonicity properties of ECAC are also invariant confluent. App state confluence is positioned between monotonicity and invariant confluence: It describes the condition under which app state is either monotonic, or only invariant confluent, i.e., can expose some form of 'time travel anomaly' depending on the order in which events become visible. We conclude that all ECAC safety properties are invariant confluent.

5 ECAC Consequences and Results

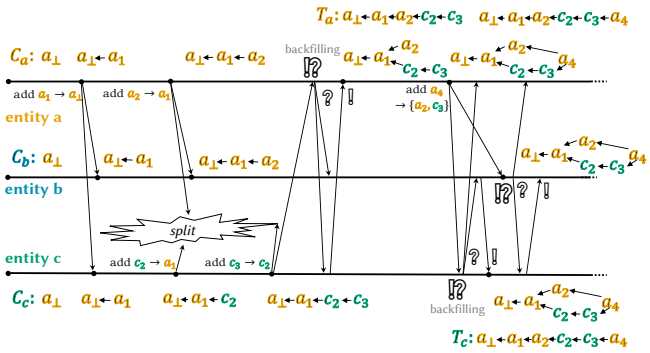
5.1 Enforceability

In his seminal work [21], Schneider states that security policies enforceable by execution monitoring must be safety properties and must be enforceable by terminating the subject to prevent the violation. As liveness properties are not enforceable by termination, they are out of scope for execution monitors and have to be ensured independently of the behavior of potentially Byzantine entities. Eventual event visibility is the only liveness property of ECAC. It is ensured by backfilling, but only under the assumption of a connected component of correct entities. For a correct entity performing an ECAC algorithm, locally created events fulfill all safety properties by definition. For remote events from other, possibly incorrect entities, all safety properties are enforceable by terminating further processing of offending events, i.e., by denying them causal or topological authorization. While Schneider's work is concerned with centralized execution monitors, he already notes the idea of decentralized execution monitors: "the security policy for a distributed system might be specified by giving a separate security automaton for each system host. Then, each host would itself implement the [...] mechanisms for only the security automata concerning that host". For enforcement by a coordination-free decentralized execution monitor, we need to combine the work of Schneider and the work of Bailis et al. on invariant confluence [3]: To be enforceable by a coordination-free decentralized execution monitor, a safety property must also be invariant confluent. As all ECAC safety properties are also invariant confluent, we conclude that ECAC safety properties are enforceable by decentralized execution monitors.

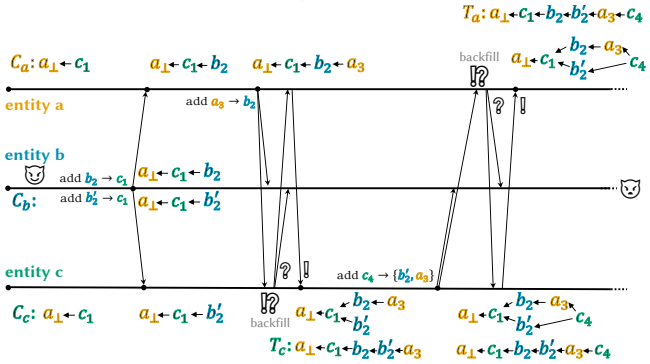
5.2 Partition, Equivocation, and Backdating

We now show the behavior of ECAC in critical scenarios, namely partition, equivocation, and backdating, where eventually consistent access control behaves differently compared to centralized access control. We assume a system $S^+ = \{a, b, c\}$ of three entities, entity b may exhibit Byzantine behavior. While coordination-based approaches are still viable with two correct and one faulty entity, this simplification is for illustration purposes: due to a's and b's autonomous decisions to their best of knowledge and belief, the assessment would be unchanged by any number of additional Byzantine entities. The key point of these scenarios depicted in Fig. 6 is to show how the causal order of events with its immutable predecessors enables immutable causal authorization under partition and Byzantine misbehavior, while the topological order enables non-monotonic revocation of authorizations to still be strongly convergent. As practical example, we take an electronic health record (EHR) stored as chronicle, featuring as entities a patient a, their health insurer b, and their general practitioner c. The EHR is replicated among all entities, to ensure availability of reads and writes without internet access. The EHR consists of medical findings and therapeutic schedules by practitioners, associated cost coverage declarations of insurers, the patient's master data, as well as the EHR authorizations, all described as events.

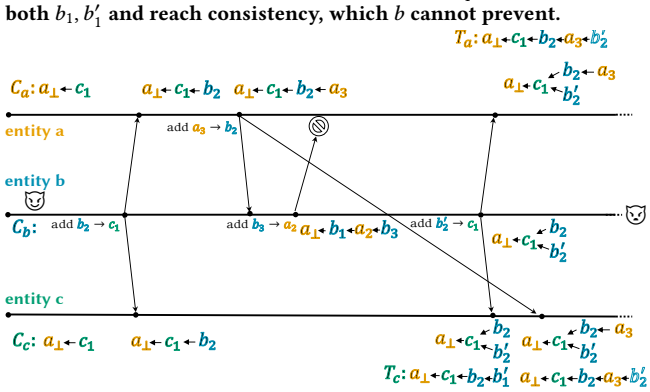
For the partition scenario displayed in Fig. 6a, there is a temporary partition between c and $\{a, b\}$, leading to events $\{a_2, c_2, c_3\}$ not reaching every entity. In the LAC model, entities would be unable to verify the authorization of affected events and reject them, i.e., be unavailable under partition. In the ECAC model, entities that received the events accept the events as causally authorized, and store them. After the partition is over, all entities eventually notice the lost events due to the references to unknown predecessors in newly-incoming events. To decide causal authorization, entities try to gather lost events by backfilling, and eventually succeed if a correct entity has seen them. For the EHR access control example, we say that event c_2 is a master data update of the patient by their general practitioner, and c_3 is a new medical finding. Event a_2 revokes the authorization of practitioner c to update master data, but still allows to add new findings. As a_2 and c_2 are sent concurrently during the partition, entity c uses its authorization to create c_2, as it has not yet heard of the revocation. Due to causal authorization immutability and eventual event visibility, entity a eventually decides



(a) Due to a network partition, entity a misses event c_2, b misses $\{c_2, c_3\}$, and c misses a_2. When a sees c_3, a cannot verify causal authorization due to the missing c_2. Entity a starts backfilling, eventually receives $\{c_2, c_3\}$, verifies causal authorization, and adds them to C_a. After b and c backfilled missing events, they are consistent with a.



(b) Byzantine entity b performs equivocation: It creates concurrent events b_1, b_1' and sends b_1 to a and b_1' to c. As both b_1, b_1' are causally authorized, b managed to create an inconsistency between a, c. Eventually, a, c exchange new events that refer to b_1, b_1'. They backfill, see both b_1, b_1' and reach consistency, which b cannot prevent.



(c) Byzantine entity b sends event b_1 which discontents a, prompting a to revoke the authorization of b to send further events in a_2. To evade revocation, b manipulates its local execution monitor to act as if it still had the necessary authorization for b_3, which fails causal authorization at a's local execution monitor. Then, b pretends to have not seen a_2, and sends a backdated event b_1' concurrent to a_2, passing causal authorization. Entities a, c cannot distinguish whether b_1' was created before or after b saw a_2. However, revocations may act against causally concurrent events on execution: assuming $a_2 \leq_L b_1'$, event b_1' does not pass topological authorization, and is not executed.

Figure 6: Partition, Equivocation, and Backdating Scenarios

c_2 as causally authorized. We assume that the lexicographic order L prioritizes authorization events, i.e., $a_2 \leq_L c_2$. Then the revocation a_2 is topologically earlier, and revokes topological authorization of c_2. Thereby, the EHR at c exhibits non-monotonic behavior: while c executed c_2 during the partition and updated the master data, it will ignore c_2 and restore the old master data as soon as it learned about a_2, i.e., $a_2 \in T_c \Rightarrow \neg z(x^{<c_2}(T_c), c_2) \Rightarrow c_2 \notin x(T_c)$. This scenario shows the effects of app state confluence, which allows non-monotonicity only if an event loses topological authorization.

For the equivocation scenario in Fig. 6b, we assume that Byzantine entity b tries to create an inconsistency between a and c by sending them different but concurrent events, i.e., b performs equivocation using events b_2, b_2'. In the LAC model, this scenario does not exist: all events are totally ordered in the order in which they become visible for the (logical) central entity, whereby there are no concurrent events. While a and c temporarily only know of one of the equivocation events, due to the same mechanics that came into play during the network partition scenario of Fig. 6a, both a and c will eventually see both b_2, b_2'. Thereby, they eventually end up with a consistent chronicle, topological order, and state set, and eventual event visibility and event set convergence are fulfilled. For the EHR access control example, assume that events b_2, b_2' are conflicting cost coverage declarations: b_2 declares cost coverage for treatment schedule c_1 to the patient a and grants a with the authorization to accept the treatment, while b_2' declares that the cost of treatment schedule c_1 are not covered, and treatment access of a is revoked. At first, a and c will report a different cost coverage status, and the practitioner would deny treatment to the patient. Based on the cost coverage b_2, a accepts the treatment in a_3. The eventual event visibility and event set convergence properties of the system ensure that eventually, a and c have causal authorization and certain knowledge and proof that insurer b concurrently sent both b_2 and b_2', and can hold b accountable for its equivocation. The concurrent changes will be executed in accordance with the lexicographic order, i.e., the resulting cost coverage depends on the lexicographically larger event of b_2, b_2' for both $\{a, c\}$. Assuming $b_2' \leq_L b_2$, the cost coverage grant is executed before the revocation, and practitioner c records treatment execution and results in c_4.

For the backdating scenario in Fig. 6c, we assume that Byzantine entity b tries to evade an authorization revocation done by a in a_3 by manipulating its local execution monitor. In the LAC model, this scenario does not exist: as all events are executed in the central order, anything sent by b after the central entity executed a_3 is subject to the revocation described by a_3. For the EHR access control example, we assume that b_2 is a positive cost coverage declaration for treatment schedule $a_\perp$, but includes a patient's contribution. Patient a is discontent with the contribution and revokes further EHR write access of insurer b, intending to switch to another insurer. Insurer b wants to send a negative cost coverage declaration now, trying to evade the revocation. Insurer b first manipulates its local execution monitor to ignore the revocation in a_3 to send the negative cost coverage b_3 anyway, which fails at the execution monitor of a as causally unauthorized. In a second attempt, b dates back negative cost coverage b_2', listing only c_1 as predecessor, thereby stating $b_2' \parallel b_2$. As b_2' is causally authorized, correct entities $\{a, c\}$ cannot differentiate whether b_2' was created after b already knew about a_3, or whether b_2' just happened to be in transit for a very long time,

Characteristic	CLAC	ECAC	Comment
Logical architecture	centralized	decentralized	CLAC simulates a centralized entity via coordination (Sec. 3)
Total event ordering	append-only	grow-only	In ECAC, new events can appear in the past (Sec. 5.2 Backdating)
Partial event ordering	\	append-only	In ECAC, only a partial order of events is append-only (Sec. 3)
Authorization decision latency	$O(d)$	$O(1)$	ECAC latency is independent of network latency d (Sec. 5.2 Partition)
Policy update latency	$O(d)$	$O(1)$	ECAC entities exert policy updates immediately (Sec. 5.2 Partition)
Authoriz. decision complexity	$O(1)$	$O(1)$	Local decision complexity is identical
Coordination complexity	$O(n^2)$	$O(1)$	CLAC coordination overhead scales with number of system entities n
Byzantine fault tolerance	$f/n < 1/3$	$f/n < 1$	ECAC tolerates any fraction of faulty entities f (Sec. 5.2 Equivocation)

Table 2: Comparison between coordination-based linearizable access control (CLAC) and eventually consistent access control (ECAC). For the same access control model, decision complexity itself is identical between CLAC and ECAC. ECAC avoids the price to pay for coordination among entities, but can only enforce invariant-confluent security policies (c.f. Section 5.1). Latency and resilience is drastically improved: The latency until an entity decides access or exerts a policy update is independent of network latency and thereby available under partition, and access control is effective even if faulty entities are the majority.

and must accept both as causally authorized. This exemplifies that entities can claim any causal predecessors with impunity, as long as the event is causally authorized — akin to the fork-join model of causality [18]. Assuming that negative cost coverage declarations are executed before positive declarations, i.e., $b_2' \leq_L b_2$, practitioner c would perform treatment under the positive declaration despite knowing both concurrent declarations. Backdating underlines the importance of the prioritization rules of causally concurrent events through their lexicographic order: assuming that $a_3 \leq_L b_2'$, event b_2' is never executed by $\{a, b\}$, as they have seen a_3 first.

Overall, the scenarios show the effect of replacing up-front coordination with subsequent reconciliation on access control: The overall system exhibits high resilience, i.e., continues to provide availability under detrimental circumstances, and can tolerate Byzantine behavior. In essence, the price to pay for the beneficial properties of eventually consistent access control is the mutability of the topological order, i.e., the execution of events to the entity's best of beliefs on the overall set of events, that takes effect on executing inherently non-monotonic actions like authorization revocations.

5.3 Result Overview

Table 2 summarizes our characterization of ECAC and contrasts it with CLAC to highlight the trade-off between improved latency, coordination, and fault tolerance at the cost of departing from a single append-only total order of actions to two orders: an append-only partial order for storage and a grow-only total order for execution. The fundamentally improved latency and fault tolerance by autonomous authorization decisions and autonomous policy updates lead to decentralized access decisions to the best of knowledge and belief, i.e., access decisions are made in the conviction of their correctness but under the condition of fallibility due to incomplete knowledge on previous and concurrent events in the system.

6 Conclusion

In this paper, we defined the ECAC model for eventually consistent access control to capture how practical systems based on Byzantine fault tolerant CRDTs, like the Matrix group communication system, break with the conventional conceptual model for access control. The explication of ECAC into properties shows that applications have to cope with some form of "time travel anomaly", which we describe as providing access control to the best of knowledge and belief. Thereby, this paper provides the necessary foundation to a formal security verification of the access control aspects of the Matrix specification and similar systems. The semantics and security notions of eventually consistent access control are highly relevant for practical, geo-distributed, resilient systems that can cope with arbitrary network and process faults. In contrast to centralized models, ECAC authorization decisions and policy updates have optimal latency and optimal fault tolerance even in a Byzantine environment. However, the systemic difference between centralized and eventually consistent access control outlined in this paper needs to be taken into account.

Acknowledgments

This work was funded by the Helmholtz Pilot Program Core Informatics.

References

[1] Paulo Sérgio Almeida. 2024. Approaches to Conflict-free Replicated Data Types. *ACM Comput. Surv.* 57, 2 (Nov. 2024), 51:1–51:36. doi:10.1145/3695249
[2] Paulo Sérgio Almeida. 2024. A Framework for Consistency Models in Distributed Systems. doi:10.48550/arXiv.2411.16355 arXiv:2411.16355
[3] Peter Bailis, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2014. Coordination Avoidance in Database Systems. *Proceedings of the VLDB Endowment* 8, 3 (Nov. 2014). doi:10.14778/2735508.2735509
[4] Davide Frey, Lucie Guillou, Michel Raynal, and François Taïani. 2023. Process-Commutative Distributed Objects: From Cryptocurrencies to Byzantine-fault-tolerant CRDTs. doi:10.48550/arXiv.2311.13936 arXiv:2311.13936 [cs]
[5] Alex Good, John Mumm, and Brooklyn Zelenka. 2025. *Keyhive: Local-first Access Control.* https://www.inkandswitch.com/keyhive/
[6] Diane Hamblen. 1986. Only the Limits of Our Imagination: An exclusive interview with RADM Grace M. Hopper. *Chips Magazine* (July 1986). https://web.archive.org/web/20090114165606/http://www.chips.navy.mil/archives/86_jul/interview.html
[7] Michael A. Harrison and Walter L. Ruzzo. 1978. Monotonic Protection Systems. In *Foundations of Secure Computation.* Atlanta, Georgia, USA.
[8] Joseph M. Hellerstein and Peter Alvaro. 2020. Keeping CALM: When Distributed Consistency Is Easy. *Commun. ACM* (Aug. 2020). doi:10.1145/3369736
[9] Maurice Herlihy. 1991. Wait-Free Synchronization. *ACM Transactions on Programming Languages and Systems* (1991). doi:10.1145/114005.102808
[10] Matthew Hodgson. 2023. Matrix 2.0. https://archive.fosdem.org/2023/schedule/event/matrix20/
[11] Florian Jacob, Luca Becker, Jan Grashöfer, and Hannes Hartenstein. 2020. Matrix Decomposition: Analysis of an Access Control Approach on Transaction-based DAGs without Finality. In *Proceedings of the 25th ACM Symposium on Access Control Models and Technologies.* ACM. doi:10.1145/3381991.3395399
[12] Florian Jacob, Carolin Beer, Norbert Henze, and Hannes Hartenstein. 2021. Analysis of the Matrix Event Graph Replicated Data Type. *IEEE Access* 9 (2021),

28317–28333. doi:10.1109/ACCESS.2021.3058576
[13] Florian Jacob and Hannes Hartenstein. 2024. Logical Clocks and Monotonicity for Byzantine-Tolerant Replicated Data Types. In *Proceedings of the 11th Workshop on Principles and Practice of Consistency for Distributed Data*. ACM, 37–43. doi:10.1145/3642976.3653034
[14] Martin Kleppmann. 2022. Making CRDTs Byzantine Fault Tolerant. In *Proceedings of the 9th Workshop on Principles and Practice of Consistency for Distributed Data*. ACM. doi:10.1145/3517209.3524042
[15] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. 2019. Local-First Software: You Own Your Data, in Spite of the Cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019)*. ACM. doi:10.1145/3359591.3359737
[16] Ninghui Li, Benjamin N. Grosof, and Joan Feigenbaum. 2003. Delegation Logic: A Logic-based Approach to Distributed Authorization. *ACM Trans. Inf. Syst. Secur.* 6, 1 (Feb. 2003), 128–171. doi:10.1145/605434.605438
[17] Steve Loynes. 2024. *UNICC Selects Element for Secure Communications*. https://element.io/blog/unicc-selects-element-for-secure-communications/
[18] Prince Mahajan. 2012. *Highly Available Storage with Minimal Trust*. Ph. D. Dissertation. University of Texas at Austin. http://hdl.handle.net/2152/16320
[19] Garson O'Toole. 2018. *It's Easier To Ask Forgiveness Than To Get Permission*. https://quoteinvestigator.com/2018/06/19/forgive/
[20] Pierre-Antoine Rault, Claudia-Lavinia Ignat, and Olivier Perrin. 2023. Access Control Based on CRDTs for Collaborative Distributed Applications. In *22nd International Conference on Trust, Security and Privacy in Computing and Communications*. doi:10.1109/TrustCom60117.2023.00187
[21] Fred B. Schneider. 2000. Enforceable Security Policies. *ACM Transactions on Information and System Security* (2000). doi:10.1145/353323.353382
[22] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. Conflict-Free Replicated Data Types. In *Stabilization, Safety, and Security of Distributed Systems*. Springer. doi:10.1007/978-3-642-24550-3_29
[23] David Spence, George Gross, Cees de Laat, Stephen Farrell, Leon HM Gommans, Pat R. Calhoun, Matt Holdrege, Betty W. de Bruijn, and John Vollbrecht. 2000. *AAA Authorization Framework*. RFC. IETF. doi:10.17487/RFC2904
[24] D.B. Terry, A.J. Demers, K. Petersen, M.J. Spreitzer, M.M. Theimer, and B.B. Welch. 1994. Session Guarantees for Weakly Consistent Replicated Data. In *Proceedings of 3rd International Conference on Parallel and Distributed Information Systems*. 140–149. doi:10.1109/PDIS.1994.331722
[25] The Matrix.org Foundation CIC. 2024. *Matrix v1.13*. Specification. https://spec.matrix.org/v1.13/
[26] D. Turner. 2004. Total Functional Programming. *JUCS - Journal of Universal Computer Science* 10, 7 (July 2004). doi:10.3217/jucs-010-07-0751
[27] Werner Vogels. 2009. Eventually Consistent. *Commun. ACM* 52, 1 (Jan. 2009), 40–44. doi:10.1145/1435417.1435432
[28] Mathias Weber, Annette Bieniusa, and Arnd Poetzsch-Heffter. 2016. Access Control for Weakly Consistent Replicated Information Systems. In *Security and Trust Management*. Springer. doi:10.1007/978-3-319-46598-2_6

A Abstract ECAC Algorithm

For a more comprehensible ECAC implementability demonstration open to scrutiny, we now describe a simple ECAC algorithm that abstracts the complex ECAC implementation of Matrix [25] (c.f. Section 2). The algorithm is based on previous work on abstracting Matrix [13] as a composition of CRDTs, but adapted to match the terms of the ECAC model. While walking through the algorithm, we show how it fulfills all ECAC model properties defined in Section 4.

In Algorithm 1, the foundation of Matrix is described as a CRDT for hash chronicles. We assume an unlimited number of Byzantine entities that participate in the CRDT, but also assume that correct entities form a connected component, i.e., cannot be stopped from communicating by Byzantine entities. The hash chronicle CRDT's state is a join-semilattice that converges by exchanging deltas, i.e., a delta-state CRDT [1, 13]. Here, the join-semilattice is defined as the set of all sets of valid events with set union as join operation, $(\mathcal{P}(\mathbb{E}), \cup)$. Specifically, the local state of a hash chronicle is the entity's set of visible events *vis*. The local state is initialized with the pre-shared genesis event $e_\perp$, as an anchor for access control that authorizes the chronicle creator to add the first membership and level assignment events.

Algorithm 1 Delta-state hash chronicle (run by each entity $s \in S$). Given are the universe of events $\mathbb{E}$, the genesis event $e_\perp$, and the recursive hash function h_r. Abstraction of [25, Server-Server API]

state visible event set $vis \subseteq \mathbb{E}$
initial $vis \leftarrow \{e_\perp\}$
query $<_C (e_1, e_2 \in C) : p \in \{\perp, \top\}$
  $p \leftarrow \exists e \in C\colon h_r(e) \in e_2.pre \wedge e_1 <_C e$
query $C\ (vis) : C \subseteq vis$
  $C \leftarrow \{e_\perp\}$ ▸ largest downward-closed subset directed at $e_\perp$
  repeat
    $C^\dagger \leftarrow \{e \in vis \setminus C \mid e.pre \subseteq \{h_r(e) \mid e \in C\}\}$
    $C^\dagger \leftarrow \{e \in C^\dagger \mid z(x^{<e}(C \cup \{e\})\}$
    $C \leftarrow C \cup C^\dagger$
  until $C^\dagger = \emptyset$
mutate $add\ (e \in \mathbb{E}) : \delta \subseteq \mathbb{E}$
  $e.pre \leftarrow \{h_r(\hat{e}) \mid \hat{e} \in \max_C(C(vis))\}$
  $\delta \leftarrow \{e\}$
on operation$(add(e))$
  $\delta = add(e)$
  $vis' \leftarrow vis \cup \delta$
  $gossip(\delta)$
on receive$(\delta \subseteq \mathbb{E})$
  if $\forall e \in \delta\colon e.pre \neq \emptyset \vee e = e_\perp$ **then**
    $vis' \leftarrow vis \cup \delta$
periodically
  $gossip(\max_C(C(vis)))$
  $request(\bigcup\{e.pre \mid e \in vis\} \setminus \{h_r(e) \mid e \in vis\})$

Algorithm 1 describes two query functions. The query function $<_C$ determines whether an event e_1 is causally earlier than another event e_2 by looking for a chain of recursive hash links from e_2 to e_1, based on the set of recursive hashes $e.pre$ of the direct causal predecessors of an event e. The query function $C(vis)$ derives the entity's current chronicle C from *vis* by traversing the set of recursive predecessor hashes $e.pre$ in reverse order, i.e., going up from $e_\perp$. The result is the largest downward-closed subset directed at $e_\perp$, which fulfills the *chronicality* property. The query also verifies the causal authorization of any event before adding it to the resulting chronicle, whereby *storage authorization* is fulfilled as well.

We now walk through how events are added and replicated in Algorithm 1. The mutate function $add(e)$ creates a δ-update from new event e by assigning e's set of direct predecessor hashes $e.pre$ with the set of maximal elements of the entity's current chronicle as timestamp. On calling $add(e)$, the entity joins *vis* with δ to apply the update, and gossips δ to all other entities. On receiving a δ, entities verify that only the genesis event has no predecessors, and add it to their visible event set. As events are immutable and only added, *vis* is grow-only. The recursive hash links in $e.pre$ unambiguously define the causal history of any event e, ensuring *causal predecessor immutability*. As *vis* is grow-only and the chronicle $C(vis)$ is a subset of *vis*, *chronicle monotonicity* is ensured. Periodically, entities gossip their maximal chronicle events, and backfill by requesting events for which they have the recursive hash, but not the event

itself. Gossiping and backfilling ensures *eventual event visibility* under the assumption of a connected component of correct entities. As $C(vis)$ as well as $T(E)$ and $x(T)$ in Algorithm 2 are total functions [26] that terminate and deterministically return the same output given the same input, *event set convergence* is fulfilled. Due to eventual event visibility and event set convergence in Byzantine environment, the algorithms represent a Byzantine-tolerant CRDT, already shown in prior work [12, 13].

Let us now discuss the functions of Algorithm 2 that build on the hash chronicle CRDT of Algorithm 1. The topological ordering function $T(E)$ performs topological sorting on the chronicle subset $E \subseteq C$. It takes the set of causally earliest, yet unsorted events first, fulfilling *causal consistency*, to then take the lexicographically earliest event, fulfilling *lexicographic consistency*. The resulting event is used as next event in the topological order, which ensures *topological totality*. Due to chronicle monotonicity and as $T(E)$ operates on chronicle subsets and only extends them with additional relations to form a total order, *topological monotonicity* is ensured. The event execution function $x(T)$ takes a totally-ordered chronicle subset of events, i.e., a result of function $T(E)$. The function walks through the total order and executes events in order. It ignores topologically unauthorized events, which ensures *execution authorization*. Topologically authorized events are added to the resulting app state X, which ensures *app state authorization*. If a topologically later event assigns an attribute to an object, it replaces the previous event for that attribute. Due to topological order execution, the only way that a topologically later event is replaced by a topologically earlier event when $T \subseteq T'$ is that the later event is ignored as topologically unauthorized, which ensures *app state confluence*.

We finally discuss the lexicographic order $<_{L(X)}$ and the authorization function $z(X)$ of this algorithm. The lexicographic order $L(X)$ defined by $<_L (X)$ orders two events e_1, e_2 based on an app state set X as returned by $x(T)$. As first criterion, the lexicographic order prioritizes authorization events, i.e., e_1 is before e_2 if e_1 is an authorization event but e_2 is not. This criterion ensures that authorization revocations are executed before concurrent non-authorization events, in order to prevent revocation evasion. The next criterion looks at the permission level of the subjects of e_1 and e_2. Events of higher-level subjects are executed first, to ensure that events by higher-level subjects are executed before any events of lower-level subjects, especially authorization revocations. The final criterion is based on the recursive hash value of the events, the event with the lower hash value is topologically earlier. The hash comparison ensures that the lexicographic order is total even in the presence of Byzantine entities: the hash function's collision resistance probabilistically ensures that this criterion always orders any two events. However, it is only the last criterion, as a Byzantine entity can easily create an event with a smaller recursive hash than the recursive hash of any given event. On every bit flip in the Byzantine event, there is a 50 % chance for the Byzantine entity that the hash is smaller than the average other event, i.e., a random sequence of $\{0, 1\}$. The authorization function $z(X, e)$ decides whether event e is authorized given the app state set X, based on the Level- and Attribute-based Access Control model [11] employed by Matrix. Event authorization is decided by four criteria: authorization for the group, the action, the object, and level, which all must be fulfilled to be authorized. The event is authorized for the communication group if the subject $e.sbj$ was declared to be `IN` the communication group by a previous action of type `mbr`. The event is authorized for its action if the action type $e.act$ is assigned with a level less or equal than the event's subject $e.sbj$. The event is authorized for its object if either it has no object, or the object is the subject, or the object is assigned with a strictly lesser permission level than the subject. Thereby, equally-leveled subjects cannot remove each other from the communication group, which avoids revocation cycles. Finally, if the event assigns levels, it is only authorized if it does not raise the level of any entity or action type o above the level of the event subject $e.sbj$, and does not lower the level of something above the subject's level.

Algorithm 2 Topological order T, lexicographic order L via $<_{L(X)}$, execution function $x(T)$, and authorization function $z(X, e)$. Abstraction of [25, Room Version 11, Server implementation].

```
function T (E ⊆ C) : T ⊇ E
    T ← ∅                                  ▷ topological ordering of chronicle subset E
    for n = 0 to |E| do
        E_min ← min_C(E)                   ▷ set of causally smallest events
        X_n ← x(T)
        e_n ← min_{L(X_n)}(E_min)          ▷ lexicograpically smallest event
        T_n ← e_n                          ▷ assign next event in topological order
        E ← E \ {e}
function x (T) : X ⊆ T
    X ← ∅                                  ▷ app state set of executing totally-ordered T
    for n = 0 to |T| do
        e ← T_n                            ▷ next event to execute in topological order
        if z(X, e) then                    ▷ ignore if topologically unauthorized
            if e.obj ≠ ⊥ then              ▷ replace previous event
                e_x ← e_x ∈ X | e_x.act = e.act ∧ e_x.obj = e.obj
                X ← (X \ {e_x}) ∪ {e}
            else
                X ← X ∪ {e}
function <_{L(X)} (e_1, e_2 ∈ 𝔼) : p ∈ {⊥, ⊤}
    lvl ← x.cnt | x ∈ X : x.act = lvl
    p ← e_1 ∈ 𝔼_z ∧ ¬e_2 ∉ 𝔼_z              ▷ prioritize authorization events
    if e_1 ∈ 𝔼_z ∧ e_2 ∈ 𝔼_z then           ▷ order by level
        p ← lvl(e_1.sbj) < lvl(e_2.sbj))
        if lvl(e_1.sbj) = lvl(e_2.sbj) then ▷ order by recursive hash
            p ← h_r(e_1) > h_r(e_2)
function z (X ∈ 𝒫(𝔼), e ∈ 𝔼) : z ∈ {⊥, ⊤}  ▷ e authorized by X?
    lvl ← x.cnt | x ∈ X : x.act = lvl
    m_sbj ← x.cnt | x ∈ X : x.act = mbr ∧ x.obj = e.sbj
    group_z ← m_sbj = IN
    action_z ← lvl(e.act) ≤ lvl(e.sbj)
    object_z ← e.obj = ⊥ ∨ e.obj = e.sbj ∨ lvl(e.obj) < lvl(e.sbj)
    level_z = ⊤
    if e.act = lvl then                    ▷ cap new levels by subject level
        lvl' ← e.cnt
        level_z ← ∀o ∈ lvl' :
                    lvl'(o) ≥ lvl(o) ⇒ lvl'(o) ≤ lvl(e.sbj)
                    lvl'(o) ≤ lvl(o) ⇒ lvl(o) < lvl(e.sbj)
    z ← group_z ∧ action_z ∧ object_z ∧ level_z
```